

Predictive models to Classify Performance of
Applications and Resource chart images

1. Model 1 : Application Performance Classification

- Classes : Efficient, Moderate or Resource-Intensive.
- Best Model found after Tuning : Support Vector Machine

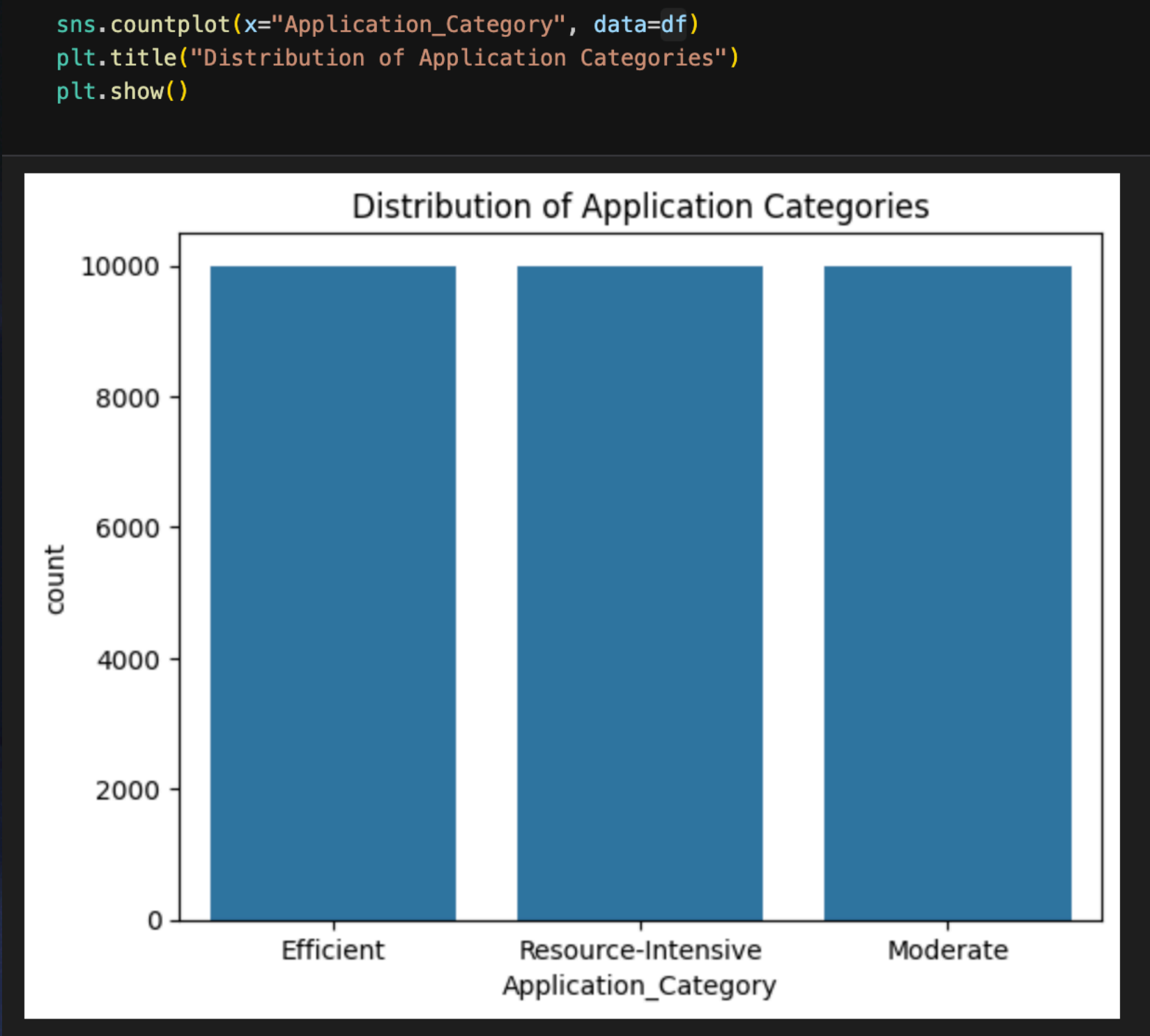
1.1 Understandig the data - EDA

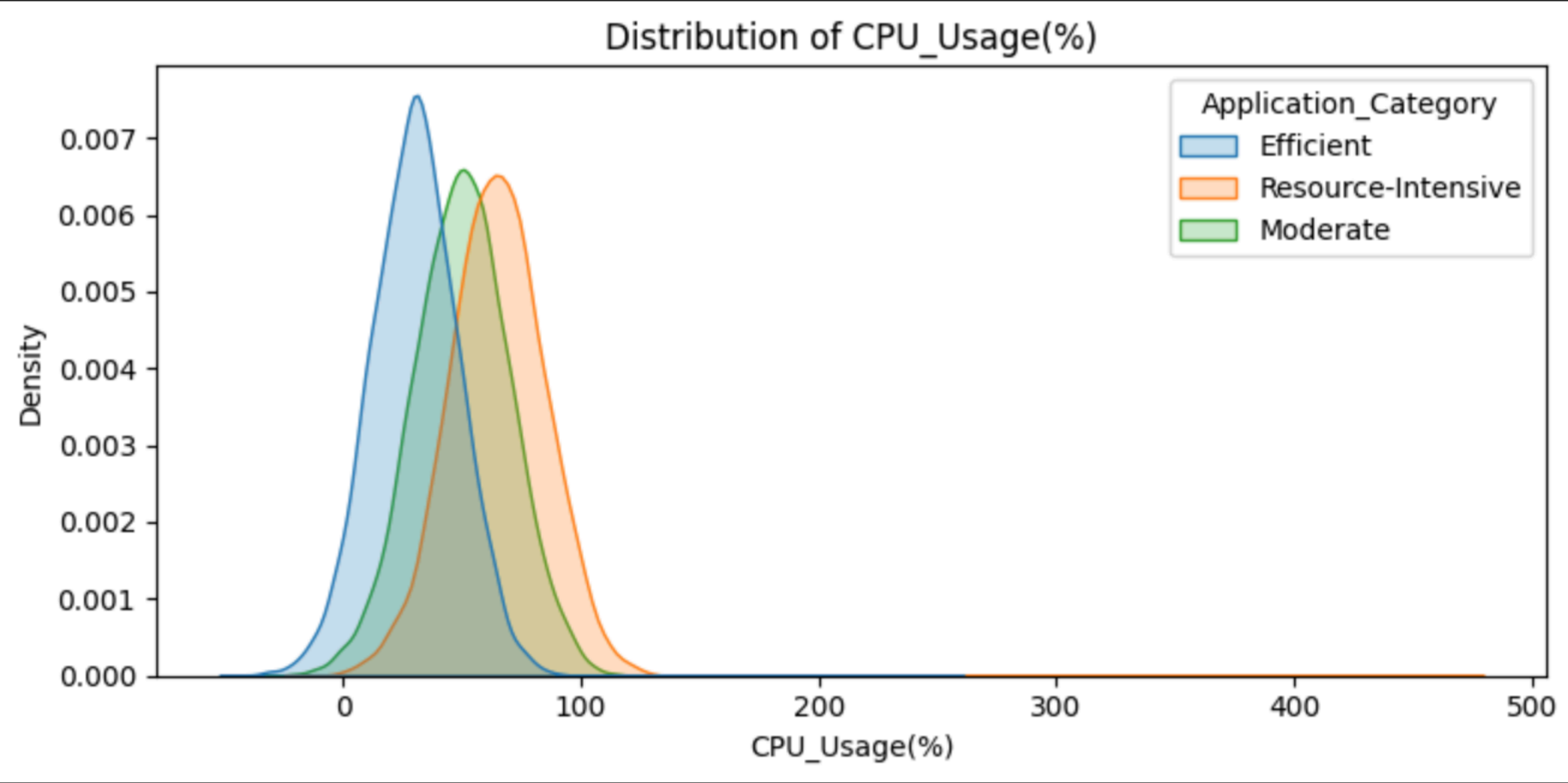
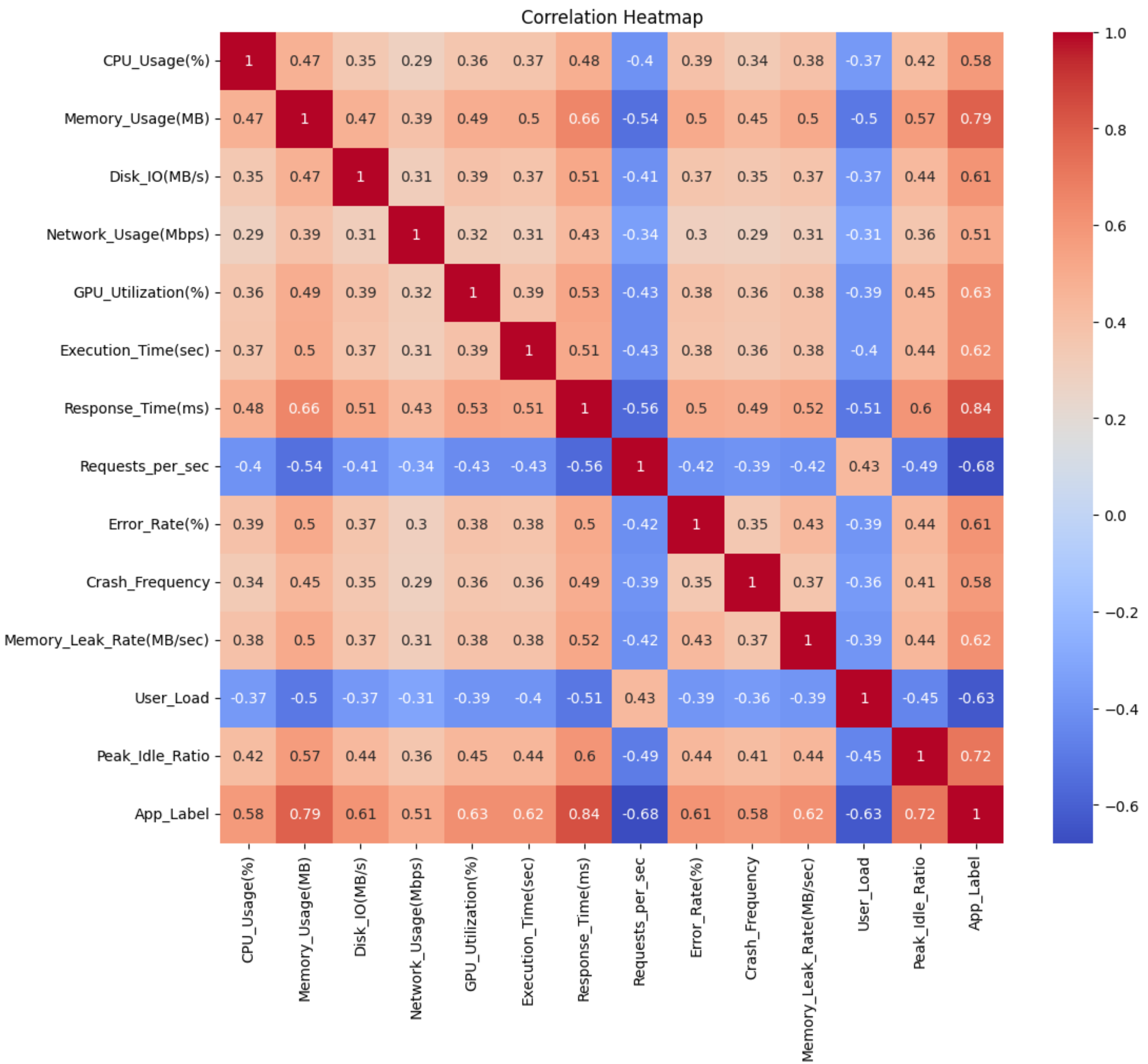
```
df = pd.read_csv(dataset_filepath)
df.head()
```

	CPU_Usage(%)	Memory_Usage(MB)	Disk_IO(MB/s)	Network_Usage(Mbps)	GPU_Utilization(%)	Execution_Time(sec)	Response_Time(ms)	Requests_per_sec	Error_Rate(%)	Crash_Frequency	Memory_Leak_Rate(MB/sec)	User_Load	Peak_Idle_Ratio	Application_Category
0	22.845913	1436.729325	46.860716	114.619811	-4.189254	5.038115	68.183387	171.929397	1.045313	0	0.380991	766.736791	0.039245	Efficient
1	64.428575	2527.157227	138.503438	140.988676	37.109380	4.501069	759.244882	71.114927	1.818975	2	0.059523	393.098830	0.482060	Resource-Intensive
2	17.300418	874.106772	99.706622	149.581220	6.266643	3.238572	214.162490	292.076324	0.780666	0	0.408419	451.424987	0.398210	Efficient
3	35.728505	2789.808555	152.446188	71.026819	42.060130	3.400988	166.093626	133.868526	3.135124	6	0.036710	452.917407	0.288462	Moderate
4	20.516290	2255.667523	77.560926	153.389839	59.404698	9.439020	89.550130	231.713756	1.860839	3	0.347140	278.414812	0.502351	Moderate

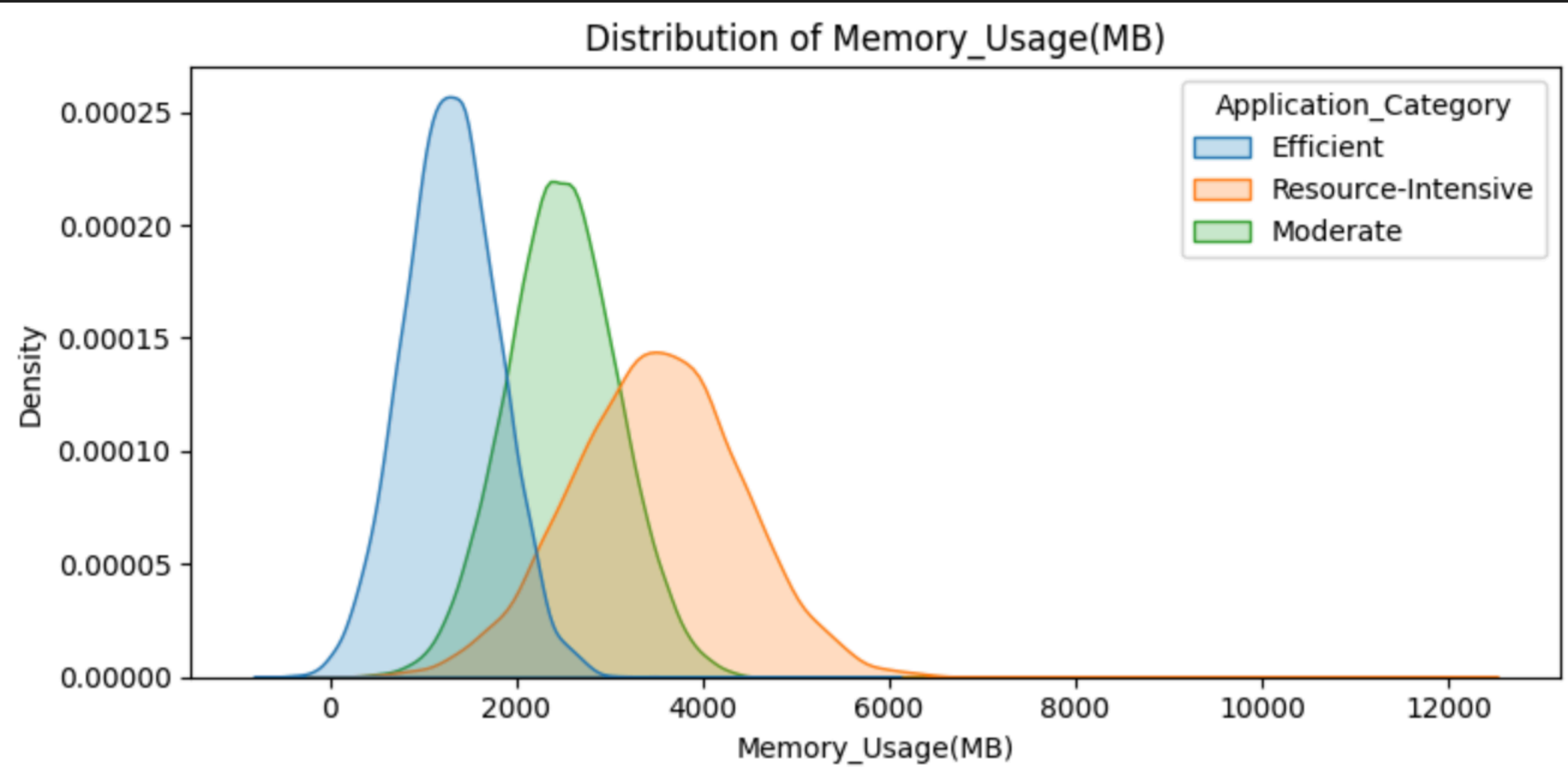
```
df.info()
```

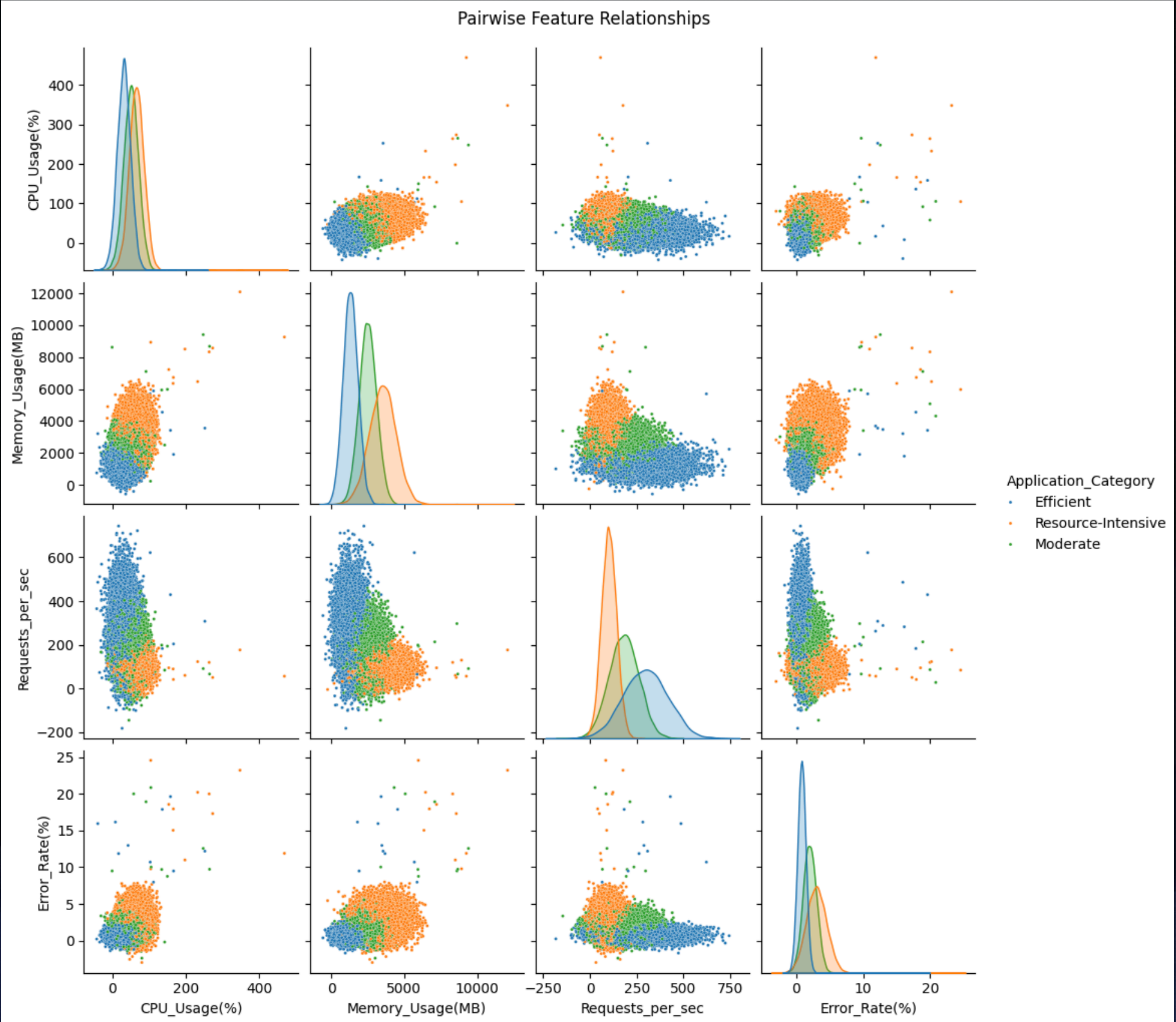
<class 'pandas.core.frame.DataFrame'>			
RangeIndex: 30000 entries, 0 to 29999			
Data columns (total 14 columns):			
#	Column	Non-Null Count	Dtype
0	CPU_Usage(%)	30000 non-null	float64
1	Memory_Usage(MB)	30000 non-null	float64
2	Disk_IO(MB/s)	30000 non-null	float64
3	Network_Usage(Mbps)	30000 non-null	float64
4	GPU_Utilization(%)	30000 non-null	float64
5	Execution_Time(sec)	30000 non-null	float64
6	Response_Time(ms)	30000 non-null	float64
7	Requests_per_sec	30000 non-null	float64
8	Error_Rate(%)	30000 non-null	float64
9	Crash_Frequency	30000 non-null	int64
10	Memory_Leak_Rate(MB/sec)	30000 non-null	float64
11	User_Load	30000 non-null	float64
12	Peak_Idle_Ratio	30000 non-null	float64
13	Application_Category	30000 non-null	object
dtypes: float64(12), int64(1), object(1)			
memory usage: 3.2+ MB			





```
Users/raghava/Documents/Projects/4-Report Generator and Load Runner/report-automation-website/lib/python3.9/s
data_subset = grouped_data.get_group(pd_key)
Users/raghava/Documents/Projects/4-Report Generator and Load Runner/report-automation-website/lib/python3.9/s
data_subset = grouped_data.get_group(pd_key)
Users/raghava/Documents/Projects/4-Report Generator and Load Runner/report-automation-website/lib/python3.9/s
data_subset = grouped_data.get_group(pd_key)
```

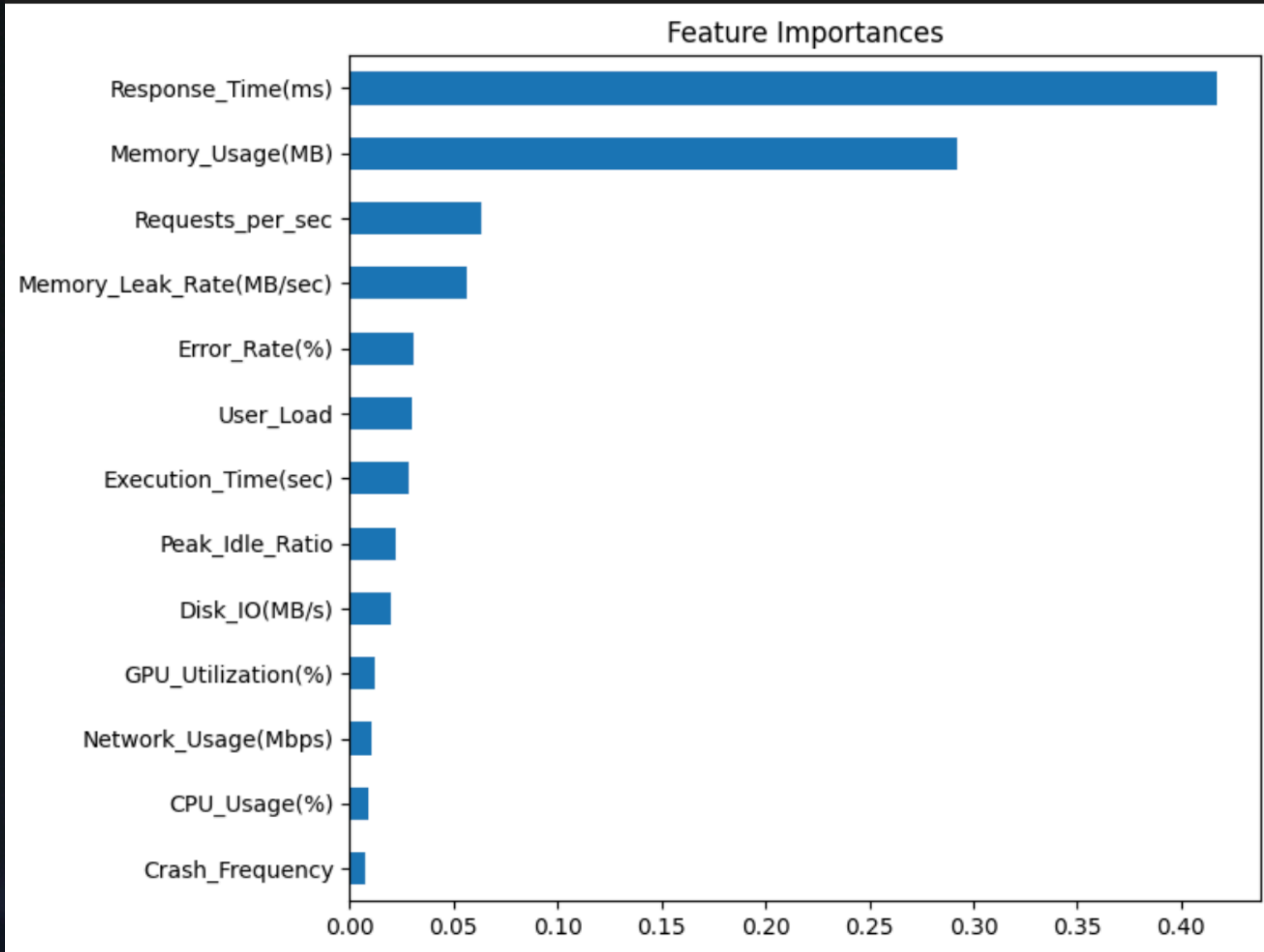




```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

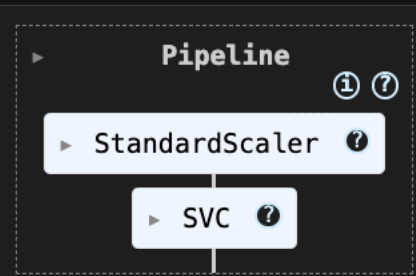
importances = pd.Series(dt_model.feature_importances_, index=X.columns)
importances.sort_values(ascending=True).plot(kind='barh', figsize=(8, 6), title="Feature Importances")
plt.tight_layout()
plt.show()
```



1.2 Model Training and Evaluation

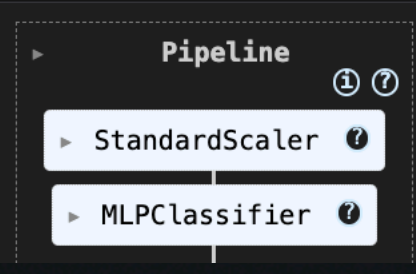
```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

svm_model = make_pipeline(StandardScaler(), SVC(kernel='rbf', probability=True, random_state=42))
svm_model.fit(X_train, y_train)
```

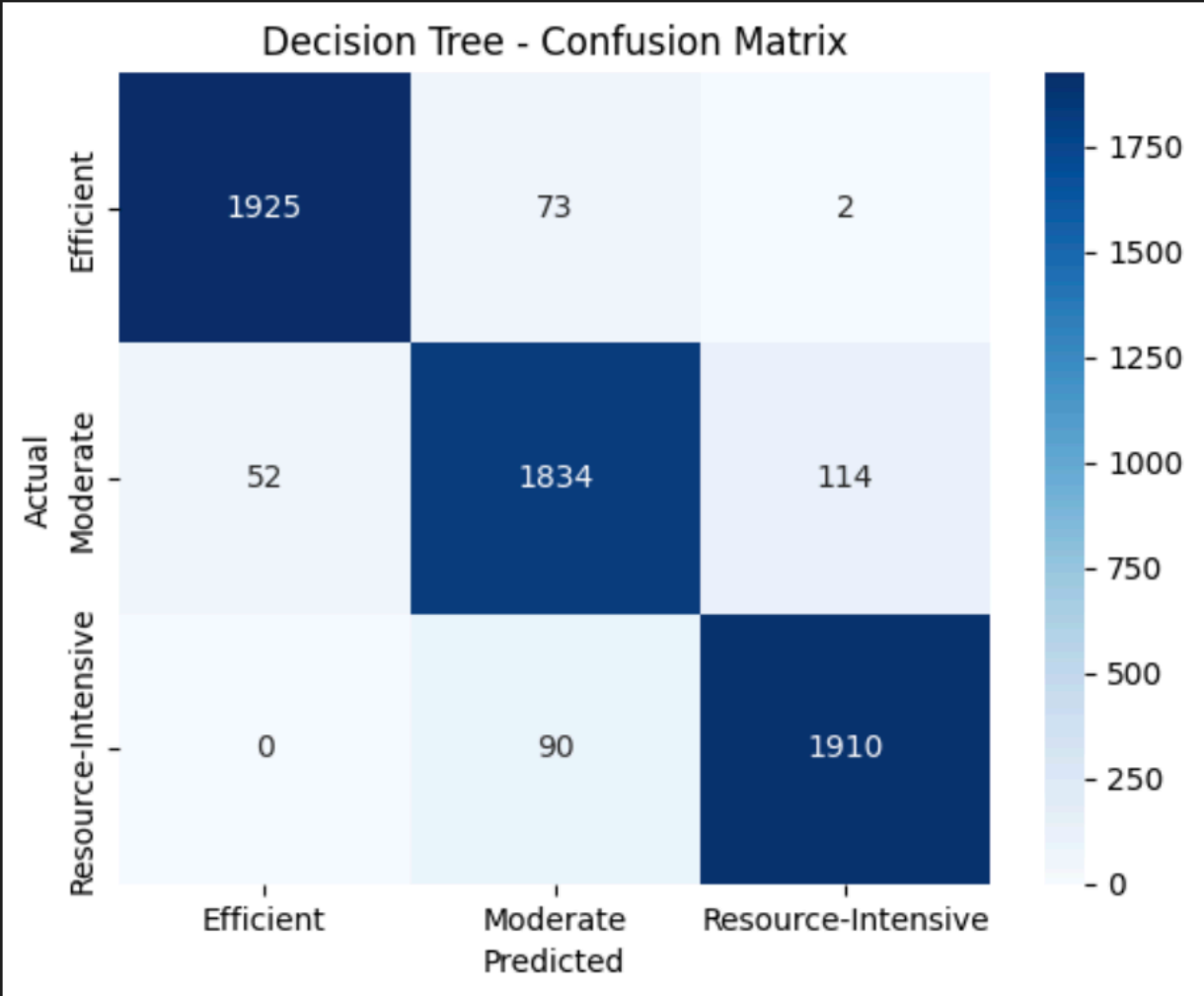


```
from sklearn.neural_network import MLPClassifier

mlp_model = make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42))
mlp_model.fit(X_train, y_train)
```



===== Decision Tree Evaluation =====				
	precision	recall	f1-score	support
Efficient	0.97	0.96	0.97	2000
Moderate	0.92	0.92	0.92	2000
Resource-Intensive	0.94	0.95	0.95	2000
accuracy			0.94	6000
macro avg	0.94	0.94	0.94	6000
weighted avg	0.94	0.94	0.94	6000



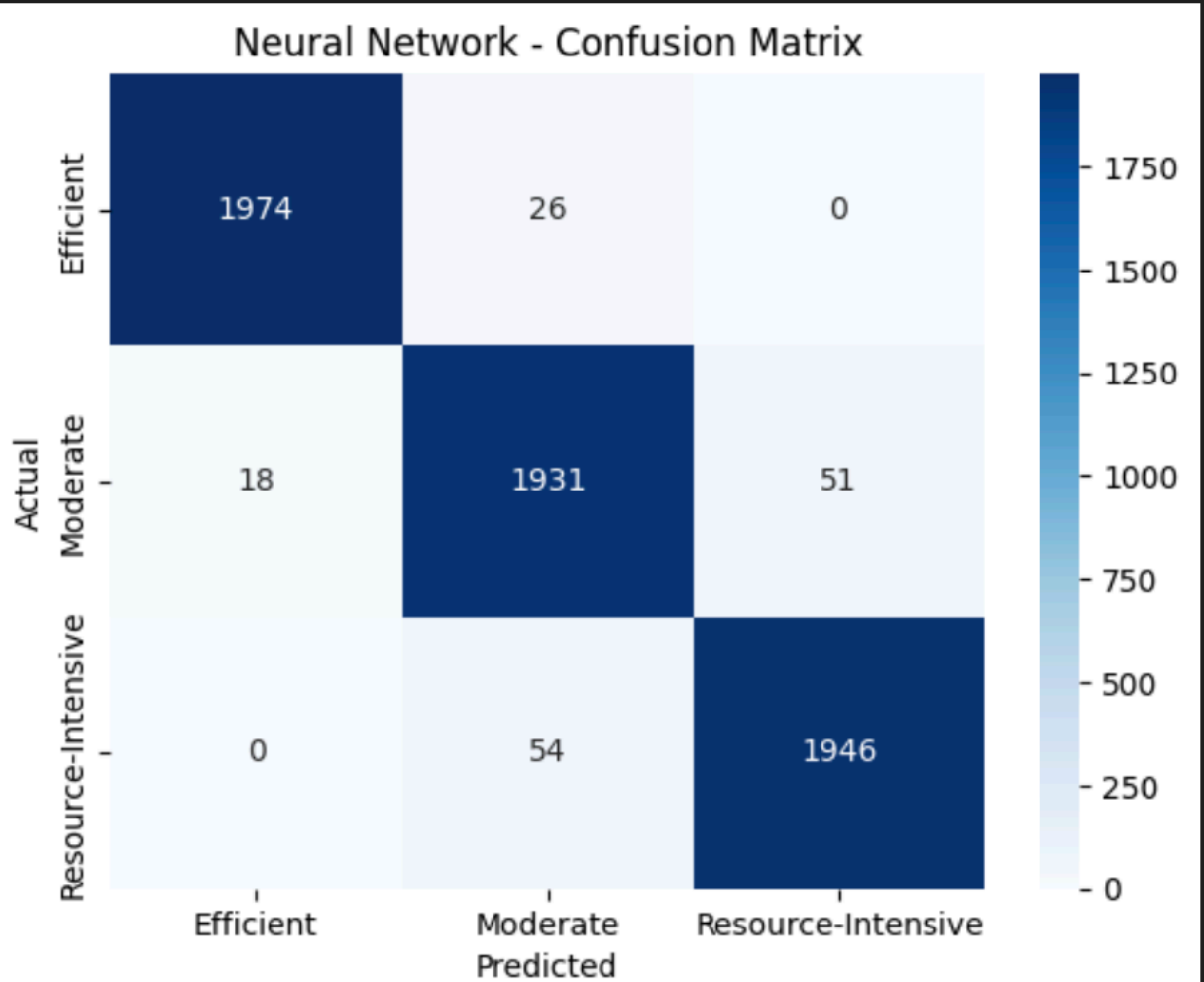
```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

def evaluate_model(model, X_test, y_test, name):
    print(f"\n===== {name} Evaluation =====")
    y_pred = model.predict(X_test)
    print(classification_report(y_test, y_pred, target_names=le.classes_))

    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

evaluate_model(dt_model, X_test, y_test, "Decision Tree")
evaluate_model(svm_model, X_test, y_test, "SVM")
evaluate_model(mlp_model, X_test, y_test, "Neural Network")
```

===== Neural Network Evaluation =====				
	precision	recall	f1-score	support
Efficient	0.99	0.99	0.99	2000
Moderate	0.96	0.97	0.96	2000
Resource-Intensive	0.97	0.97	0.97	2000
accuracy			0.98	6000
macro avg	0.98	0.98	0.98	6000
weighted avg	0.98	0.98	0.98	6000



===== SVM Evaluation =====				
	precision	recall	f1-score	support
Efficient	0.99	0.99	0.99	2000
Moderate	0.97	0.98	0.98	2000
Resource-Intensive	0.99	0.98	0.98	2000
accuracy			0.98	6000
macro avg	0.98	0.98	0.98	6000
weighted avg	0.98	0.98	0.98	6000



1.3 Model Deployment and Prediction

```
import joblib

# Save the best model – choose based on accuracy/F1
joblib.dump(svm_model, "app_perf_classifier_model.joblib")
joblib.dump(le, "app_classify_label_encoder.joblib") # save label encoder too

import joblib

# Load the saved model and label encoder
model = joblib.load("app_perf_classifier_model.joblib")
label_encoder = joblib.load("app_classify_label_encoder.joblib")

new_data = {
    "CPU_Usage(%)": 45,
    "Memory_Usage(MB)": 1800,
    "Disk_IO(MB/s)": 60,
    "Network_Usage(Mbps)": 90,
    "GPU_Utilization(%)": 25,
    "Execution_Time(sec)": 4,
    "Response_Time(ms)": 250,
    "Requests_per_sec": 180,
    "Error_Rate(%)": 1.2,
    "Crash_Frequency": 0.5,
    "Memory_Leak_Rate(MB/sec)": 0.4,
    "User_Load": 320,
    "Peak_Idle_Ratio": 0.35
}

import pandas as pd

new_df = pd.DataFrame([new_data])

# Predict numerical class
predicted_class = model.predict(new_df)[0]

# Convert to readable label
predicted_label = label_encoder.inverse_transform([predicted_class])[0]

print("Predicted Application Category:", predicted_label)
```

Predicted Application Category: Efficient

```
proba = model.predict_proba(new_df)[0]
for cls, p in zip(label_encoder.classes_, proba):
    print(f"{cls}: {p:.2f}")

Efficient: 0.62
Moderate: 0.38
Resource-Intensive: 0.00

print(proba)

[6.18725817e-01 3.81215350e-01 5.88329324e-05]

# Define class score mapping
score_mapping = {
    "Efficient": 0.0,
    "Moderate": 0.5,
    "Resource-Intensive": 1.0
}

# Get probabilities for each class
proba = model.predict_proba(new_df)[0]

# Map labels to numeric scores
class_labels = label_encoder.classes_
class_scores = [score_mapping[label] for label in class_labels]

# Calculate weighted score
final_score = sum(p * s for p, s in zip(proba, class_scores))

# Predicted label (for display)
predicted_label = label_encoder.inverse_transform([model.predict(new_df)[0]])[0]

# Output
print(f"Predicted Category: {predicted_label}")
print(f"Weighted Performance Score: {final_score:.2f}")

Predicted Category: Efficient
Weighted Performance Score: 0.19
```


2. Model 2 : Resource Chart Classification

- Classes : leak, no issue or spikes.
- Best Model found after Tuning : Convolution Neural Network

2.1 Understanding the data



2.2 Model Building

```
# Image size close to original aspect ratio (2:1 ratio like 1783x883)

datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    validation_split=0.2,
    width_shift_range=0.2,
    height_shift_range=0.2,
)

train_generator = datagen.flow_from_directory(
    data_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='training'
)

val_generator = datagen.flow_from_directory(
    data_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation'
)

model = Sequential([
    # First Conv Block
    Conv2D(32, (3, 3), activation='relu', input_shape=(256, 128, 3)), # input shape: (height, width, channels)
    MaxPooling2D(pool_size=(2, 2)),

    # Second Conv Block
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    # Third Conv Block
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    # Flatten and Fully Connected Layers
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(3, activation='softmax') # change 3 to the number of classes in your case
])

# Compile the model with a lower learning rate for better convergence
model.compile(optimizer=Adam(learning_rate=0.0005), loss='categorical_crossentropy', metrics=['accuracy'])
```

model.summary()

✓ 0.0s

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 127, 63, 32)	0
conv2d_1 (Conv2D)	(None, 125, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 62, 30, 64)	0
conv2d_2 (Conv2D)	(None, 60, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 30, 14, 128)	0
flatten (Flatten)	(None, 53760)	0
dense (Dense)	(None, 128)	6,881,408
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 6,975,043 (26.61 MB)

Trainable params: 6,975,043 (26.61 MB)

Non-trainable params: 0 (0.00 B)

2.3 Model Training

```
print(train_generator.class_indices)

# Invert the dictionary to get index-to-name mapping
index_to_class = {v: k for k, v in train_generator.class_indices.items()}

# Optional: print as a list
classes = [index_to_class[i] for i in range(len(index_to_class))]
print(classes)

{'leak': 0, 'no_issue': 1, 'spikes': 2}
['leak', 'no_issue', 'spikes']
```

```
# Train the model
model.fit(train_generator, validation_data=val_generator, epochs=10)

Epoch 1/10
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class
self._warn_if_super_not_called()
90/90 ━━━━━━━━━━━ 14s 120ms/step - accuracy: 0.4340 - loss: 1.1314 - val_accuracy: 0.7833 - val_loss: 0.5341
Epoch 2/10
90/90 ━━━━━━━━━━━ 18s 110ms/step - accuracy: 0.7887 - loss: 0.5231 - val_accuracy: 0.8444 - val_loss: 0.3773
Epoch 3/10
90/90 ━━━━━━━━━━━ 10s 113ms/step - accuracy: 0.8445 - loss: 0.4535 - val_accuracy: 0.8444 - val_loss: 0.4284
Epoch 4/10
90/90 ━━━━━━━━━━━ 10s 111ms/step - accuracy: 0.8774 - loss: 0.3408 - val_accuracy: 0.9389 - val_loss: 0.2305
Epoch 5/10
90/90 ━━━━━━━━━━━ 10s 110ms/step - accuracy: 0.8985 - loss: 0.3278 - val_accuracy: 0.9167 - val_loss: 0.2158
Epoch 6/10
90/90 ━━━━━━━━━━━ 9s 100ms/step - accuracy: 0.8793 - loss: 0.3024 - val_accuracy: 0.9222 - val_loss: 0.2081
Epoch 7/10
90/90 ━━━━━━━━━━━ 10s 109ms/step - accuracy: 0.8984 - loss: 0.2278 - val_accuracy: 0.9500 - val_loss: 0.1691
Epoch 8/10
90/90 ━━━━━━━━━━━ 10s 109ms/step - accuracy: 0.9408 - loss: 0.1738 - val_accuracy: 0.9722 - val_loss: 0.1088
Epoch 9/10
90/90 ━━━━━━━━━━━ 10s 109ms/step - accuracy: 0.9405 - loss: 0.2032 - val_accuracy: 0.9278 - val_loss: 0.1740
Epoch 10/10
90/90 ━━━━━━━━━━━ 10s 108ms/step - accuracy: 0.9479 - loss: 0.1692 - val_accuracy: 0.9611 - val_loss: 0.1136
```


2.4 Model Prediction

```
# Save the model
model.save(model_name)
```

Python

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format via `model.save(filepath, save\_format='tf')`.

```
# Set your image dimensions here (must match model input)

def predict(image_path, loaded_model):
    def preprocess_image(image_path):
        img = cv2.imread(image_path) # Load the image
        # if img is None:
        #     return f"cv2 failed to read the image. Path exists? {os.path.exists(image_path)}. path : {image_path} Exact path: {repr(image_path)}"
        # else:
        #     # return f"Image loaded successfully. Shape: {img.shape}"
        img = cv2.resize(img, (img_width, img_height)) # Resize to (width, height)
        img = img / 255.0 # Normalize pixel values
        img = np.expand_dims(img, axis=0) # Add batch dimension
        return img

    processed_image = preprocess_image(image_path)

    prediction = loaded_model.predict(processed_image)
    print(prediction)
    class_index = np.argmax(prediction)

    predicted_label = classes[class_index]

    return_string = f"Predicted Class: {predicted_label}.\nConfidence {prediction[0][class_index]*100:.2f}%\n"
    # for i, label in enumerate(classes):
    #     return_string += f"{label}: {prediction[0][i]*100:.2f}% confidence\n"

    return return_string
```

Python

```
loaded_model = load_model(model_name)
print(predict("/content/charts_classification_training_images/leak/leak_100.png", loaded_model))
```

Python

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

1/1 0s 311ms/step

[[1.000000e+00 6.290833e-20 4.130827e-12]]

Predicted Class: leak.

Confidence 100.00%